

Чисти функции, състояние и странични ефекти



Функционално програмиране чрез обикновени C# примери

Какво трябва да стане ясно

1

Състояние

2

Страничен ефект

3

Чиста функция

Програмата пази информация в даден момент.

Част от тази информация може да се променя при изпълнение.

Страничен ефект има, когато функцията прави наблюдаема промяна извън своя резултат.

Чиста функция: един и същ вход → един и същ резултат, без наблюдаем страничен ефект.

Локални промени вътре във функцията не са проблем, ако не се виждат отвън.

1. Състояние

Състояние = информацията, която програмата пази в даден момент.

Примери за състояние

```
int score = 20;  
bool gameOver = false;  
int currentPlayer = 2;  
List<int> numbers = new List<int>();
```

Преди → действие → след

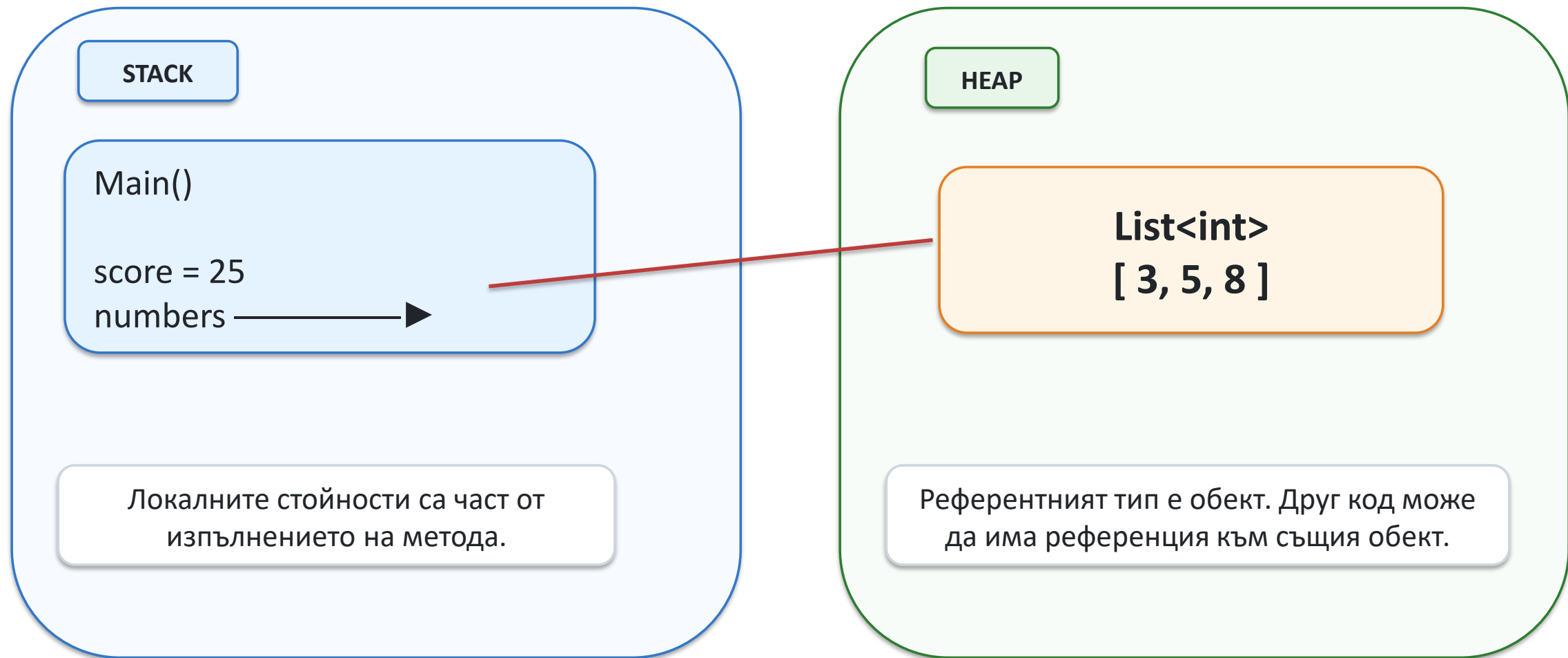
score = 20

+ 5

score = 25

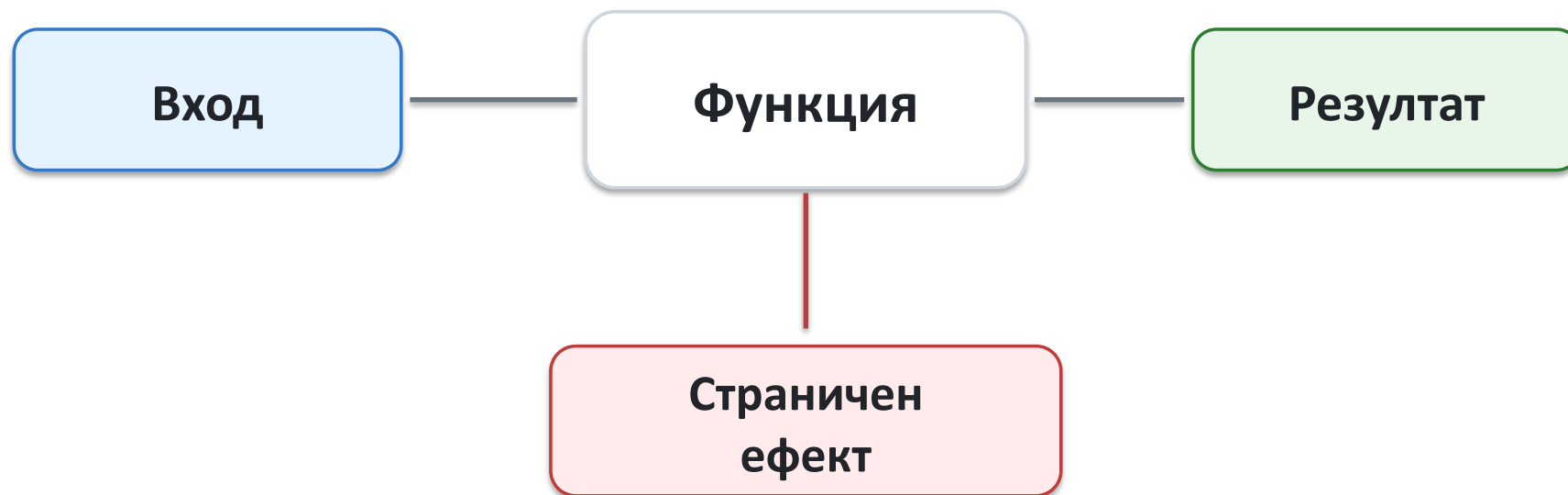
Когато стойност или обект се промени, казваме, че се е променило състояние.

Състояние в паметта: локална стойност и обект



2. Страничен ефект

Страничен ефект = наблюдаема промяна извън стойността, която функцията връща.



променя обект или глобална променлива • пише на конзола • чете/пише файл • използва Random

Примери за странични ефекти

Промяна на обект

```
static void AddNumber(  
    List<int> numbers, int n)  
{  
    numbers.Add(n);  
}
```

След извикване списъкът е различен.
Това може да бъде видяно от друг код.

Писане на конзолата

```
static void PrintNumber(int n)  
{  
    Console.WriteLine(n);  
}
```

Функцията не връща стойност, но
променя външния свят: появява се текст.

Резултатът зависи от скрито външно състояние

```
static int score = 65;  
static string GetResult()  
{  
    return score >= 50  
        ? "Успех!": "Опитай отново!";  
}
```

Резултатът на функцията зависи от
изменяема променлива

3. Чиста функция

Чиста функция

Пример

```
static bool IsEven(int number)
{
    return number % 2 == 0;
}
```

IsEven(8) → true

IsEven(8) → true

Два критерия

1. Един и същ вход → един и същ резултат.
2. Няма наблюдаем страничен ефект.

Нищо извън функцията не се променя.

Обикновени примери за чисти функции

Double

```
static int Double(int x)
{
    return x * 2;
}
```

Double(5) → 10

Max

```
static int Max(int a, int b)
{
    return a > b ? a : b;
}
```

Max(3, 7) → 7

IsValidAge

```
static bool IsValidAge(int age)
{
    return age >= 0 && age <= 120;
}
```

IsValidAge(15) → true

Тези функции изчисляват резултат. Не променят външен обект, файл, конзола или глобална стойност.

Локална изменяемост ≠ страничен ефект

Вътре sum се променя

```
static int SumNumbers(int[] numbers)
{
    int sum = 0;

    foreach (int number in numbers)
    {
        sum += number;
    }

    return sum;
}
```

Няма проблем

`sum` е локална променлива.

Другите методи не виждат междинните ѝ стойности.

Входният масив не се променя.

Резултатът зависи само от входа.

SumNumbers([2, 5, 8]) → 15

Кога изменяемостта става проблем?

Локална промяна

```
static int Calculate(int x)
{
    int result = x * 2;
    result += 5;
    return result;
}
```

След return локалната променлива не е видима отвън.

Наблюдаема промяна

```
static void IncreaseScore(Player player)
{
    player.Score++;
}
```

Обектът Player е променен и друг код може да го види.

Нечисти функции

Random

```
static int RollDice()
{
    return Random.Shared.Next(1, 7);
}
```

Няма вход, но може да върне различен резултат.

Глобално / споделяно състояние

```
static int counter = 0;

static void Increment()
{
    counter++;
}
```

Функцията променя стойност, която не е нейна локална променлива.

Връзка с конкурентното програмиране

Локални стойности

Thread 1
stack
int x = 5

Thread 2
stack
int x = 10

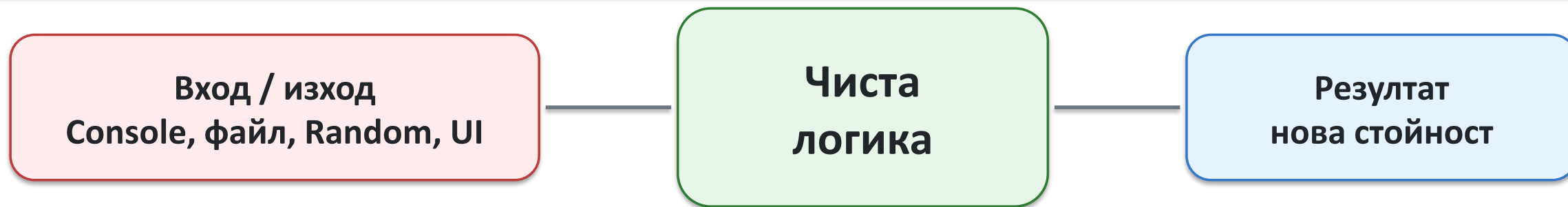
Локалните променливи на различни
нишки не си пречат.

Споделен обект

HEAP
Player
Score

Споделеното изменяемо състояние
може да доведе до race condition.

Практически принцип



Входът и изходът почти винаги имат странични ефекти.

Основните изчисления могат да бъдат чисти функции.

Когато страничните ефекти са отделени, кодът се чете и тества по-лесно.

Функционалният стил не забранява състоянието, а го прави по-контролирано.

Упражнение: чиста или нечиста? Защо?

A

```
static int Square(int x)
{
    return x * x;
}
```

един и същ вход → един и същ резултат

B

```
static void Print(int x)
{
    Console.WriteLine(x);
}
```

пише на конзолата

C

```
static int CountPositive(int[] a)
{
    return a.Count(x => x > 0);
}
```

само чете масива и връща брой

D

```
static void AddOne(List<int> a)
{
    a.Add(1);
}
```

променя подадения списък

E

```
static int Sum(int[] a)
{
    int s = 0;
    foreach (int x in a) s += x;
    return s;
}
```

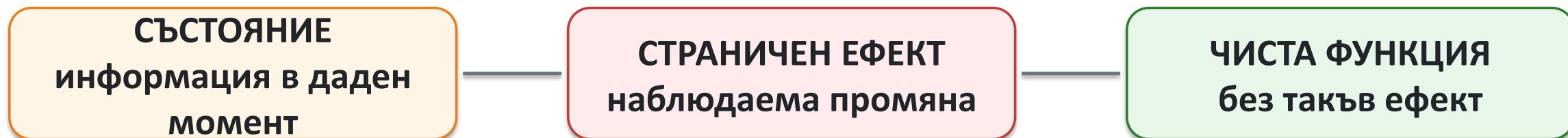
има локална промяна, но тя не се вижда отвън

F

```
static int GetRandom()
{
    return Random.Shared.Next();
}
```

резултатът зависи от Random

Обобщение



Чистата функция зависи само от своите входни данни.

Страничният ефект е промяна, която може да се наблюдава извън върнатия резултат.

Локална променлива може да се променя вътре във функцията.

Проблемът е споделеното или външно наблюдаемо изменяемо състояние.

Функционалният стил се стреми да отделя чистата логика от страничните ефекти.